Chapter 2 Solutions

CH02-Q01

Register numbers are `$t0=8`, `$t1=9`, `$s0=16`, and `$sp=29`.

```
sll: op=000000 rs=00000 rt=01001 rd=01000 shamt=00011 funct=000000
```

$$I = (9 \ll 16) + (8 \ll 11) + (3 \ll 6) = 0x000940c0.$$

`addiu` has opcode 9 and immediate −16 encoded as `0xfff0`.

$$I = (9 \ll 26) + (29 \ll 21) + (16 \ll 16) + 0xfff0 = 0x27b0fff0.$$

The first is R-format, with unsigned shift amount 3. The second is I-format. Its immediate sign-extends to `0xfffffff0` despite the `u` suffix. Each encoding occupies four bytes.

CH02-Q02

The unsigned value of `0xfffffff6` is 4294967286. Subtract 2^{32} to obtain signed −10.

The low bits of `0x7fffffff + 1` are `0x80000000`. Positive operands have produced a negative sign bit, so signed overflow occurs. A trapping `add` raises an exception rather than committing a valid signed result.

`0xffffffff + 1` produces low bits `0x00000000` and carry out 1. This is unsigned overflow. Signed $-1 + 1$ does not overflow.

Use `sltu` for unsigned values, where `0xffffffff` exceeds 1. Use `slt` for signed values, where −1 is less than 1. Hexadecimal notation does not determine the intended interpretation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CH02-Q03

Let $i = R[t0]$. At each body entry,

$$0 \le i \le n, \qquad R[v0] \equiv \sum_{k=0}^{i-1} |a[k]| \pmod{2^{32}}.$$

This invariant applies to elements whose `sub` completes; the minimum integer is the exceptional case below. Initialization sets `i=0` and sum 0. If `i<n`, the initial `beq` falls through after its `nop`, processes `a[i]`, and increments to `i+1`. At `i=n`, it executes the slot and exits. The body therefore runs at indices $0, 1, \ldots, n-1$, exactly `n` times. For `n=0`, it runs zero times.

For `a=[-5,3]`:

Body index	Loaded `$t2`	Updated `$v0`	Next `$t0`
0	-5	5	1
1	3	8	2

The next test sees `2=2` and exits.

`0x80000000` represents -2^{31}. `sub $t2,$zero,$t2` requires $+2^{31}$, which exceeds the signed 32-bit range. It raises signed overflow and does not write the result. `subu` would avoid the trap but produce `0x80000000`, not a representable positive signed value. Choose saturation, a wider representation, or explicit error handling.

`addu` wraps modulo 2^{32} without a signed-overflow trap. Exact accumulation needs a wider accumulator, carry checks, or restricted inputs. It cannot repair an earlier `sub` exception.

CH02-Q04

`outer` must preserve its caller's `$s0`, because it changes that callee-saved register, and its incoming `$ra`, because a nested `jal` overwrites it. It must also preserve any caller-saved value still live after `inner`. The sole incoming argument can be held in `$s0`; unspecified `$s1`/`$s2` values cannot supply additional arguments.

Reserve offsets 0..15 as the outgoing argument home area, 20..23 for an additional local, 24..27 for old `$s0`, and 28..31 for incoming `$ra`. An initially eight-byte-aligned `sp` remains aligned after subtracting 32.

```
addiu $sp, $sp, -32
sw    $ra, 28($sp)
sw    $s0, 24($sp)
addu  $s0, $a0, $zero
addu  $a0, $s0, $zero
jal   inner
nop
```

Any calculation before the call must use defined inputs. `jal` gives `inner` the current call's `PC+8`. The callee preserves `$s0`; it may overwrite `$t` and `$a` registers. Use its `$v0` result before restoring and returning:

Lin Cing computer-science-self-study

```
lw    $s0, 24($sp)
lw    $ra, 28($sp)
addiu $sp, $sp, 32
jr    $ra
nop
```

Loads use the allocated base and precede release. Their order need not reverse the saves when addresses and data dependencies are correct. Simultaneously live values need distinct slots.

Two sequential calls reuse this frame and outgoing area. Save the incoming `$ra` once; each `jal` supplies its own continuation to `inner`. If the first `$v0` must survive the second call, store it in a distinct slot such as `20(sp)`, or another preserved callee-saved register. Enlarge the frame only if more live storage is required. An `inner` that fails to restore `$s0` violates its callee-save obligation. `outer` must still restore its own caller's value before returning.

CH02-Q05

`0xfffa` sign-extends to −6. Its byte displacement is $-6 \times 4 = -24 = -0x18$.

$$target = 0x00400010 - 0x18 = 0x003ffff8.$$

The J-format target has upper nibble zero from `PC+4`.

$$target = (0 \ll 28) \vee (0x0004000 \ll 2) = 0x00010000.$$

`0x1abcdef4` has upper nibble 1 and is unreachable directly by that instruction. A register-indirect transfer or reachable trampoline can cross regions. Branch immediate 2 denotes eight bytes.

CH02-Q06

Ordinary load/add/store is not an atomic read-modify-write. Both cores can read `x` and each write `x+1`, losing an increment. With LL/SC retry, each successfully completed conditional store counts as one increment. If both cores eventually succeed once, the counter increases twice modulo 2^{32}.

`ll` reads and establishes a reservation. `sc` uses the old `rt` value as store data. On failure it leaves shared memory unwritten and sets `rt=0`; on success it stores and sets `rt=1`. Retry must execute a fresh `ll`, not increment the status value. Another core may have changed memory, so failure does not imply memory still equals the value originally read.

The question stipulates that the reservation has been invalidated. Therefore `sc` fails, makes no shared-memory write, and returns 0; another core may nevertheless have changed memory. Which events invalidate a reservation and the monitored granule depend on the ISA version and platform.

Atomicity does not imply starvation freedom. One core may repeatedly lose. Progress requires additional conditions such as fair scheduling, contention limits, or a backoff policy with a stated progress argument; it cannot be inferred from an atomic instruction alone.

CH02-Q07

The assembler emits local labels, encodings, symbols, and relocations, leaving external references unresolved. The static linker resolves `dot` while combining the objects. It records the shared-library dependency and dynamic linkage for `axpy`. The dynamic loader maps the library and performs eager or lazy symbol resolution.

A relocation specifies how symbol value, addend, and location modify a field. Otherwise the linker cannot distinguish a numeric constant from an address that needs adjustment.

File offsets count bytes in a file. Virtual addresses identify process locations. Segment mappings connect ranges with different origins and alignment.

A finite PC-relative field may not reach a distant library. A nearby stub or linkage entry followed by an indirect transfer can solve this. PLT and GOT are platform-specific examples rather than universal MIPS ABI names.

CH02-Q08

$$T_A = \frac{1.8 \times 10^9 \times 1.20}{3.0 \times 10^9} = 0.720\ \text{s}, \quad T_B = \frac{1.5 \times 10^9 \times 1.55}{3.0 \times 10^9} = 0.775\ \text{s}.$$

$S_{B/A} = 0.720/0.775 \approx 0.929$. B takes 7.64% more time. Its count falls by $1/6$, while CPI rises by about 29.2%. The product determines total time.

At break-even,

$$f_B = \frac{1.5 \times 10^9 \times 1.55}{0.720} \approx 3.22917\ \text{GHz}.$$

This assumes constant CPI. Real frequency changes can alter stalls, power, and thermal behavior.

CH02-Q09

1. `op=0x23=35` identifies I-format `lw`; `rs=29=$sp`, `rt=8=$t0`, and `imm16=12`. It loads from $R[sp] + 0x0000000c$ modulo 2^{32}, with a four-byte-aligned effective address.
2. `op=4,rs=8,rt=9,imm16=0xfffc=-4` decodes as `beq $t0,$t1,-4`. `PCnext=0x00400024` and
$$target = 0x00400024 + (-4 \times 4) = 0x00400014.$$

Taken PCs are `0x00400020,0x00400024,0x00400014`; not-taken PCs are `0x00400020,0x00400024,0x00400028`. The common middle address is the slot.

3. `op=3,target26=0x100140` gives low target bits `0x00400500`. `PCnext=0x00400074` contributes high nibble zero. Target is `0x00400500`, the slot is `0x00400074`, and `$ra=0x00400078`.
4. Treating `-4` as bytes wrongly gives `0x00400020`. The actual offset first shifts left by two.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CH02-Q10

The fields occupy 0..7, 8..23, and 24..31. Maximum alignment is eight, so size is 32 bytes and `vals` starts at 8:

$$0x3000 + 5(32) + 8 + 3(4) = 0x3000 + 160 + 8 + 12 = 0x30b4.$$

```
movslq %esi, %rsi
salq   $5, %rdi
leaq   8(%rdi,%rsi,4), %rax
leaq   (%rdx,%rax), %rax
```

The first `lea` computes offset `0xb4`; the second adds base `0x3000`. Register address arithmetic sends no memory-read request.

A single-element prefix is already sorted. At `i=1,j=1`, compare `3>1`, swap to `[1,3,2]`, and decrement `j` to 0. At `i=2,j=2`, compare `3>2` and swap to `[1,2,3]`; at `j=1`, `1>2` is false and the loop stops.

Seven versus five counts equivalent bodies, tests, and one branch delay slot. It excludes prologue, initial guard, final exit control, cache misses, mispredictions, and exceptions. Strength reduction replaces `i*4` with a pointer increment; induction-variable elimination can hoist fixed-base addition. Both source forms may then execute the same instruction sequence.

References and attribution

Lin Cing, handwritten computer organization notes. Patterson and Hennessy, Computer Organization and Design, revised fourth edition, Chapter 2. CSO course slides, Chapter 2.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study